DA7067 Computational Mathematics
Exercise sheet on Machine Learning

Department of Mathematics
Stockholm University

- **Deadline:** Sunday 2026-10-11 at 23:59
- **Hand in:** upload a .zip file to the course webpage containing a PDF report summarising your results and your code in a separate file. Use around 1–2 pages for the report.
- **Oral examination:** at most 7 minutes presentation, followed by 3 minutes of questions.
- **Collaboration:** individual task. You may discuss ideas, but your analysis and conclusions must be your own.

Your task is to apply supervised learning to a dataset of your own choice. In the lecture we briefly covered linear regression, logistic regression and k-nearest neighbours (kNN) regression and classification, along with the bias–variance trade-off. Now you will see how these methods behave on real data. Everyone follows the same steps, but each of you works with your own data.

The main question is: *how well can you predict something you are curious about, and how does model complexity affect the result?*

1. **Choose data and a question.** Find a dataset with roughly 100–10 000 rows and at least 4 numeric features. I encourage you to pick different datasets from each other, as this makes the presentations more interesting for everyone. Good sources are the datasets built into scikit-learn, the UCI Machine Learning Repository and Kaggle. If your data has missing values or non-numeric columns, it is fine to simply drop those rows and columns.

2. **Frame the problem.** Is it regression or classification? What are the features and what is the label? What makes it supervised learning, and why is it a machine learning problem rather than one for classical programming?

3. **Baseline and two models.** Split the data into a training set and a test set. Compare a baseline model (a classifier that always predicts the most common class, or a regression model that always predicts the mean) to linear or logistic regression, and kNN classification or regression. Since kNN is based on distances, standardise your features before applying it. Report test performance (Root Mean Squared Error (RMSE) or Accuracy). Which method performs better on your task and why do you think that is?

4. **Bias–variance trade-off.** Apply kNN for $k = 1, 2, \ldots, 50$ and plot both the training error and the test error against k. The training error is obtained by letting the model predict the points in the training set itself. Where does the model overfit and where does it underfit? Which k would you choose, and why?

5. **Reflect.** What would you trust your model to do, and what would you not trust it with? What surprised you, if anything?

Getting started. If you have not used train/test splits or scikit-learn before, read the short tutorial `https://scikit-learn.org/stable/getting_started.html`. I recommend Python with `pandas`, `matplotlib` and `scikit-learn`. You can also use R if you are more comfortable with that. You do not have to implement the methods yourself; you are free to use built-in functions, for example `KNeighborsClassifier` in `scikit-learn` to do kNN classification, `StandardScaler` to standardise features, and `DummyClassifier` or `DummyRegressor` for the baseline.